PRD: Smart House Annotation App

Product Requirements Document for the developer. Version 1.0, 12 November 2026.

1. Overview

A lightweight web app (tablet and phone friendly) used by participants in a smart-house experiment to **annotate their own activities in real time**. Each participant, while living in the house, repeatedly records where they are, what they are doing, and how they are doing it (alone or with others). Annotations are stored with a timestamp and can be exported as CSV after the session.

Fixed: the screen flow and the CSV output. **Configurable:** the content of four JSON files (scenario, locations, activities, activity-location mapping). The app must never hard-code any location, activity or person name.

2. Goals and non-goals

Goals

- Fast annotation, ideally 4 taps plus 1 confirmation.
- Accurate timestamps, always visible to the participant.
- All options generated from the JSON files, so the researcher can change the experiment without code changes.
- Data saved to a database after every submission, and exportable as CSV (individual or whole session).

Non-goals (v1)

- No analytics or dashboards.
- No sensor integration.
- No user accounts or passwords (see Open Questions).

3. Users

- **Participant** (Person_A, Person_B, Person_C, and so on): annotates their own activity.
- **Researcher**: starts and ends the session, downloads CSV files, edits the JSON files.

4. Configuration files (may change)

The app loads these at start-up (or on session start) and builds all UI from them.

File	Purpose
`scenario.json`	Number of residents and their role names, e.g. `{"resident number": 3, "roles": ["Person_A","Person_B","Person_C"]}`
`locations.json`	List of locations (id, name, description) shown on Screen 1
`activities.json`	List of all activities (id, name)
`activities_location.json`	Maps each activity to a location (id, name, zone) so Screen 2 shows only the activities valid for the chosen location

Requirements

- The number and names of persons come only from `scenario.json`.
- Buttons on every screen are generated dynamically, and the grid layout adapts to the number of items (2 columns, as in the sketches).
- Validate the files on load and show a clear error to the researcher if one is invalid (see data issues in section 9).
- Store the loaded config version or hash with each session so results stay traceable if the files change later.

5. Global UI element (every screen)

Clock at the top of every screen, showing the current time with seconds (HH:MM:SS), updating every second. The brief lists "time" and "time with seconds" as two items; I read this as one clock showing seconds. Please confirm (Open Question 1).

Every screen after Screen 1 also shows a read-only summary of the choices made so far, plus a **Back** button.

6. Screen flow (fixed)

Screen 1: Choose location

- Clock.
- Grid of all locations from `locations.json` (sketch 1: Kitchen, Dining, Living, Bathroom, Bedroom, Stairs, Toilet, Landing, Front Door, Back Door).
- Action: tap one location, which goes to Screen 2.

Screen 2: Choose activity

- Clock.
- Chosen location (displayed).
- Grid of activities available in that location, from `activities_location.json`.
- Action: tap one activity, which goes to Screen 3.

Screen 3: Choose interaction

- Clock, chosen location, chosen activity.
- Three options: **Solo**, **Collaborative**, **Assistive**.
- Action: tap one. Solo goes to Screen 5; Collaborative and Assistive go to Screen 4 (assumption, see Open Question 3).

Screen 4: Choose person

- Clock, chosen location, activity and interaction.
- Options are the **other** residents from `scenario.json` (everyone except the current participant). Example: if the user is Person_A, show Person_B and Person_C.
- Action: tap one person. A multi-select may be needed if more than one person is involved (Open Question 4).

Screen 5: Review and submit

(The brief labels this "Interface 4" a second time; I treat it as Screen 5.)

- Clock.
- Summary of all selections: location, activity, interaction, person.
- Actions: **Edit** (jump back to the chosen step; later steps are kept or re-validated if an earlier choice changed) or **Accept/Submit**.

- On submit: save the record, show a short confirmation, then return to Screen 1 for the next annotation.

7. Session and participant handling

1. **Session start (researcher):** creates a session ID, loads the config.
2. **Participant identification:** before Screen 1, the participant picks their role (Person_A, Person_B, and so on). This is a one-time step per device and is remembered until the session ends.
3. **Session end:** the researcher ends the session, which locks it and enables downloads.

Multiple participants may use the app at the same time on different devices within the same session.

8. Data storage and export

Database: every submitted annotation is written to the database immediately (a background update with current data), so nothing is lost if a device is closed or loses connection. If offline, queue locally and sync when back online.

Record fields (one row per annotation)

Field	Example
session_id	S001
participant	Person_A
annotation_id	17
timestamp_submitted	2026-11-12 14:20:30 (ISO 8601, with timezone)
timestamp_started	time Screen 1 was opened
location_id / location	2 / living
activity_id / activity	17 / resting on the couch
interaction	solo / collaborative / assistive
other_person	Person_B (empty for solo)
edited	true/false (whether Edit was used)

Field	Example
config_version	hash or date of the JSON files used

CSV download (after the session ends)

- **Individual:** one CSV for one participant.
- **All participants:** one combined CSV for the whole session (e.g. all 3 persons in one file, sorted by timestamp, with the `participant` column).
- File names, e.g. `S001_Person_A.csv` and `S001_all.csv`. UTF-8, comma-separated, header row included.

9. Data issues found in the current JSON files (please resolve)

1. **`scenario.json` is invalid JSON:** it has a trailing comma after the roles array. The parser should either tolerate it or the file should be fixed.
2. **Naming mismatches between files:** `locations.json` has "landing area" and "bedroom 1" (space), while `activities_location.json` uses zone "landing" and "bedroom_1" (underscore). Matching by name will fail. **Recommendation:** link by `location_id` instead of a name string.
3. **Activities that belong to two locations:** the brief's table lists "Entering/Exiting house" under both Front door and Back door, but `activities_location.json` maps "entering" only to Front door and "exiting" only to Back door. The file structure (one activity, one zone) also cannot express an activity in several locations. **Recommendation:** let each activity list several location IDs, or have one row per activity-location pair.
4. **Wording differences:** "Ascending stairs" (brief) vs "climbing stairs" (JSON); "Relaxing on couch" (brief) vs "resting on the couch" / "resting or working on the couch" (JSON; the two JSON files differ from each other).
5. **Sketch 2** shows living-room activities "TV", "Vacuum", "Sit on couch" and "Window". "Vacuum" does not exist in the JSON files, and "Window" probably corresponds to "ventilating living". Confirm the final list.
6. Several `description` fields in `locations.json` are empty. Decide whether they should be shown to participants.

10. Non-functional requirements

- Responsive, touch-first design with large tap targets, usable one-handed on a phone.
- Clock accuracy: use the server time or sync with it, so timestamps are consistent across devices.
- Annotation should take under 10 seconds.
- Works on current Chrome and Safari (mobile and desktop).
- Participant data is kept private. Only the researcher can download CSVs.

11. Acceptance criteria

- Changing any of the four JSON files changes the options shown, with no code change.
- The clock with seconds is visible on all 5 screens.
- Screen 2 only lists activities valid for the chosen location.
- Screen 4 never lists the current participant.
- Submitting writes a row to the database within 2 seconds (or queues it when offline).
- Both CSV download modes work and the combined file contains all participants of the session.

12. Open questions for the product owner

1. Is it one clock with seconds, or two separate clocks (time and time with seconds)?
2. How does a participant identify themselves (pick a role on first launch, or a link per person)?
3. Should "Solo" skip the person screen? Should the interaction types be configurable in JSON too?
4. Can more than one other person be selected on Screen 4?
5. Is an annotation a point in time, or does it need a start and end time for each activity?
6. Which database is preferred, and where is it hosted?
7. After submit, does the participant always return to Screen 1, or is there an "Same as last time" shortcut?